

Redes Neuronales

U4 Tópicos Avanzados

Héctor Maravillo

Section 1

Aspects of Topological Specifications for Multilayer Neural Networks

Introduction

The most suitable Multilayer Neural Network topology to map a particular problem is usually done empirically, since the design of the network depends on:

- The adopted learning algorithm.
- The initialization of the weight matrices.
- The complexity of the problem.
- The spatial distribution of the training samples
- The quality of the training set available.

Cross-Validation methods

The **cross-validation method** is one of the most used statistical methods for selecting the best MLP topology.

Its purpose is to **evaluate** the performance of each **topology** when using a data set different from that used in the training process

Random subsampling cross-validation method

The first method is called **random subsampling cross-validation**, where the entire data set (samples) available is randomly divided into two subsets:

- **Training.** It will be used to train all candidate topologies.
- **Test (validation).** It is only applied for selecting the topology that provides better generalization results.

Random subsampling cross-validation method

As the samples from the test subset do not participate in the learning process, it is possible to **evaluate** the generalization performance of each **candidate topology** by comparing the results produced by their outputs with the desired values when using the test subset.

In practical terms, from the total data set available, around 60–90% of its elements are chosen at random for the training subset.

Random subsampling cross-validation method

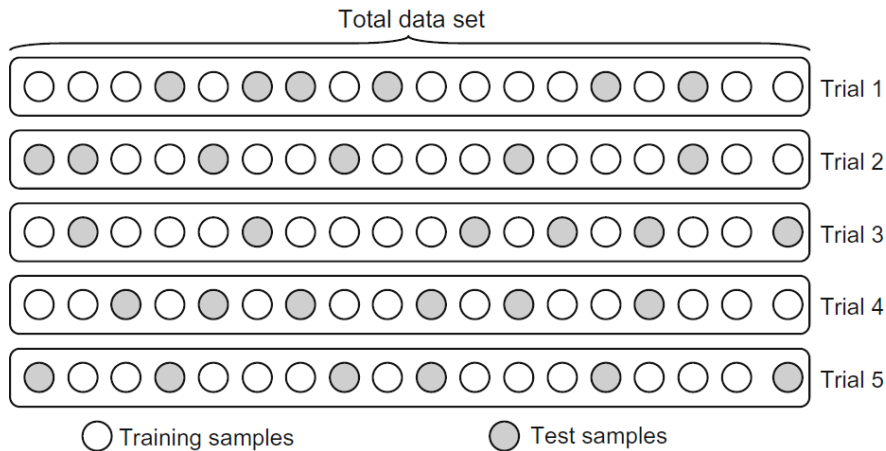


Figure: Random subsampling cross-validation method

k -fold cross-validation method

The **k -fold cross-validation** consists of **dividing** the total sample set in k partitions, where $(k - 1)$ partitions will be used to compose the training subset, and the remaining partition will be used to compose the test subset.

After this division, the learning process **is repeated** k times until all partitions have been used as a test subset.

The value of the parameter k is linked to the total number of available samples, and it is usually defined between 5 and 10.

The **global performance** of each candidate topology is obtained by evaluating the **average** of the individual performances at each one of the k trials.

k-fold cross-validation method

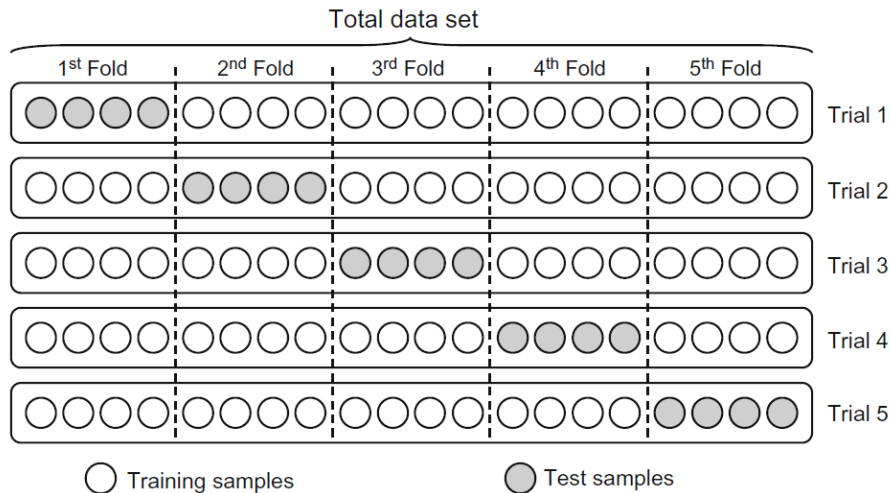


Figure: *k*-fold cross-validation method

Leave-one-out cross-validation method

The third method is called **leave-one-out cross-validation**, which consists of using a **single sample** for the **test subset**, while the other samples are used in the training subset.

The learning process is then repeated until all samples are individually used as the test subset.

This last technique is a **particular case** of the k -fold method when the parameter k assumes the value corresponding to the total number of samples.

However, there is a **high computational effort** in this case, because the learning process will be repeated, for each candidate topology, as many times as the size of the complete sample set.

General Cross-Validation Algorithm

- ➊ Define the candidate topologies for the given problem.
- ➋ Acquire the training and test subsets.
- ➌ Apply the learning algorithm to all candidate topologies using the training subsets.
- ➍ Apply the test subset to the (already trained) candidate topologies in order to evaluate their generalization potential.
- ➎ Obtain the final performance metric of each candidate topology with respect to the number of trials.
- ➏ Select the candidate topology that obtained the best global performance.
- ➐ If the global performance of the best candidate topology is within the precision required by the problem.
 - Then: End the cross-validation process.
 - Else: Specify a new set of candidate topologies and go back to **step 3**.

Leave-one-out cross-validation method

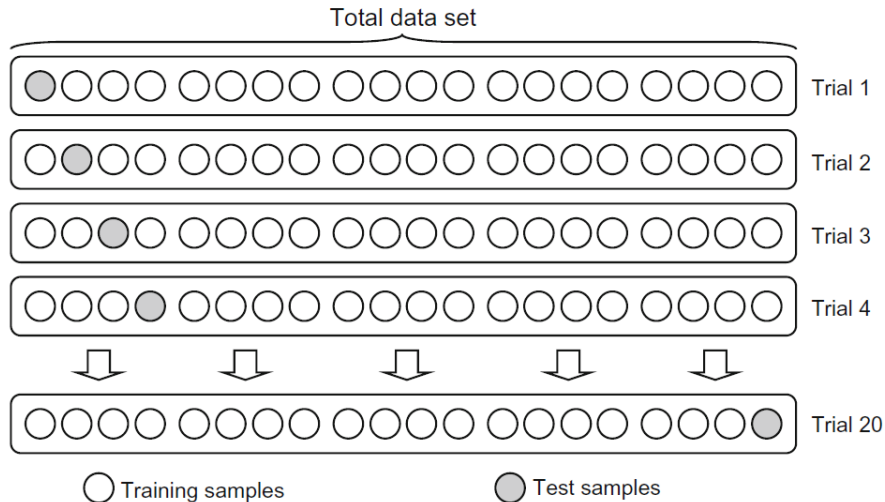


Figure: Leave-one-out cross-validation method

Incorrect validation

In contrast to all the three methods, when **all available samples** are used both for learning and validation, it becomes **impossible to verify** the generalization capability of the network.

What would be evaluated is its capability of satisfactorily **memorize** the desired responses with the respect to the presented patterns.

Alternatives

Alternatively, instead of using cross-validation methods, two **rules** are widely used for specifying a topology of an Multilayer NN constituted by a **single hidden layer**.

These rules are given by:

- **Kolmogorov method**

$$n_1 = 2n + 1$$

- **Fletcher-Gloss method**

$$2\sqrt{n} + n_2 \leq n_1 \leq 2n + 1$$

where n is the number of inputs of the network, n_1 is the number of neurons in the hidden layer, and n_2 is the number of neurons in the output layer.

Alternatives

For problems of **pattern classification** with n_c classes, another heuristic that have been usually used is that implemented by the **WEKA platform** (**Waikato Environment for Knowledge Analysis**), that is

$$n_1 = \frac{n + n_c}{2}$$

These networks, although largely used, are appropriate for just a few **well-behaved classes of a problem**, since they do not consider attributes that are, in fact, very relevant for specifying the topology of Multilayer NN.

Aspects of Overfitting and Underfitting Scenarios

It is important to highlight that increasing indiscriminately the **number of neurons**, as well as increasing the **number of intermediate layers**, **does not ensure** the **appropriate generalization** of the Multilayer Perceptron with respect to the samples of the test subset.

Overfitting condition

Invariably, such premature actions usually drive the MLP to **overfit**, a condition in which the network **memorizes** the responses to the input stimuli.

In this scenario,

- The **squared error** of the **learning process** tends to be **very low**.
- During the **generalization stage**, when the test subset is presented to the network, the squared error tends to be **very high**.

This fact is known as the **overfitting condition**.

Overfitting condition

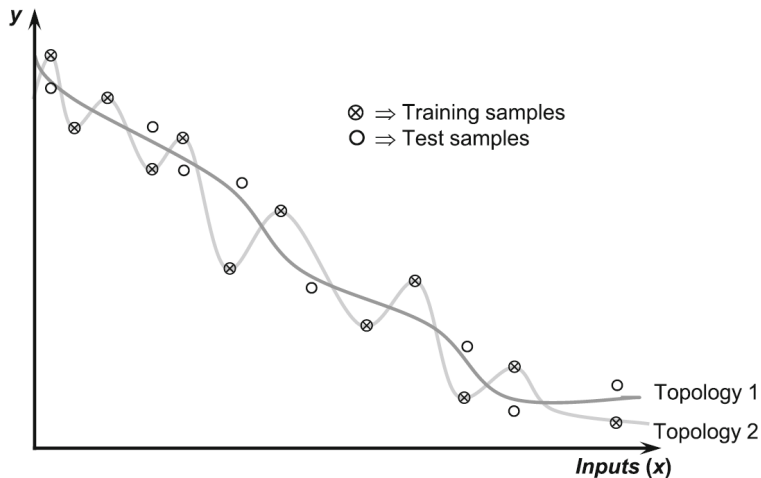


Figure: Behavior of MLP networks operation with (topology 2) and without overfitting (topology 1).

Underfitting

On the other hand, an Multilayer topology with a **reduced number of neurons** might be **insufficient** for extracting and storing features of the process.

Therefore, the network will not be able to build hypotheses about the process behavior, which results in a situation of natural **debility** called **underfitting**.

In this case, the **squared error** in both learning and test stages are **very significant**.

Aspects of Early Stopping

A simple procedure that can be used alongside with cross-validation methods is the **early** or **anticipated stopping**, in which the learning process of a **candidate topology** is **continuously monitored**, and the performance of the network is evaluated with respect to the **samples** of the **test subset**.

The training process is **stopped** when the mean squared error begins to **increase** (when the network is evaluated with the test subset) between successive epochs.

In fact, that **abrupt variation** indicates the beginning of **overfitting** of the training subset.

Overfitting condition

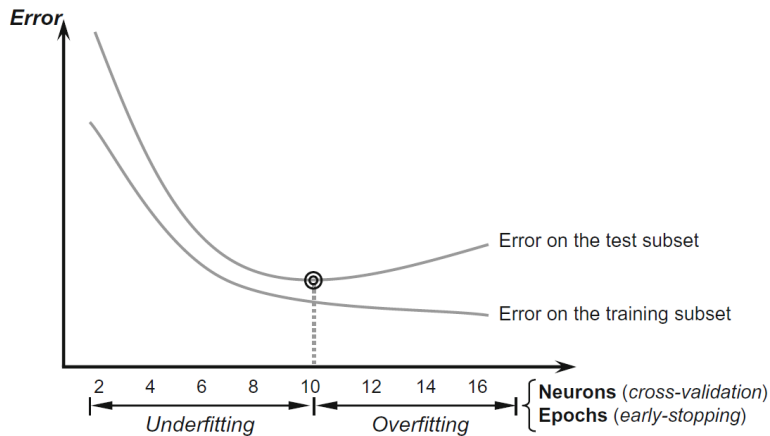


Figure: Underfitting and overfitting examples.

Implementation aspects of Multilayer Perceptron Networks

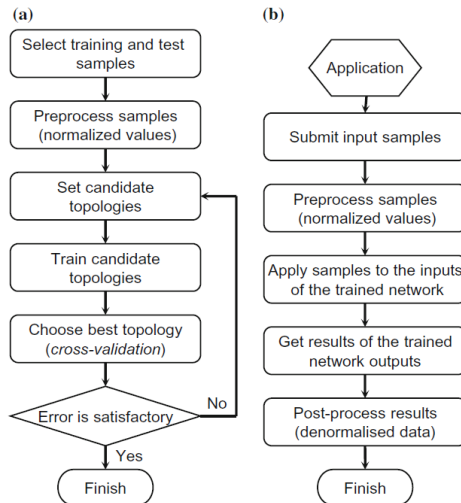


Figure: Block diagram of the training and operation stages. a Training stage. b Operation stage.

Implementation aspects of Multilayer Perceptron Networks

Practical remarks:

- Increasing the number of neurons or layers of the Multilayer Perceptron Network does not directly result in enhancements of its generalization ability.
- Considering two MLP topologies that result in the **same generalization precision**, always select the network composed of **fewer neurons**, since the network was able to extract more knowledge.
- Allocate to the **training set** all samples containing the **minimum and maximum values** with respect to any input or output variable of the MLP.

Implementation aspects of Multilayer Perceptron Networks

- **Perform the training** of each candidate topology **several times**, assuming random values for the weight matrices, in order to escape from regions of local minima.
- Initialize all weight matrices $W_{ji}^{(L)}$ with **small random values**. This recommendation comes from the analysis performed by Yan LeCun (1989). The values must be within the range of

$$-\frac{2.4}{N} \leq W_{ji}^{(L)} \leq \frac{2.4}{N}$$

where N is the number of inputs of the Multilayer Perceptron. The strategy tries **to avoid** that the weighted sum of the input values **saturates the activation functions**.

Implementation aspects of Multilayer Perceptron Networks

- Associate suitable values to the **learning terms** α , which must be within the ranges of

$$0.05 \leq \alpha \leq 0.75$$

- Impose a **maximum number of epochs** as an **additional stop criterion** for the MLP training because it is a simpler and efficient strategy for stopping the training when the specified precision ϵ becomes unachievable.
- **Normalize** the input and output samples to avoid the saturation domains of activation functions.
- Adopt the **hyperbolic tangent** as **activation function** for the neurons of the hidden layers. This function contributes to enhancing the convergence process of the network.

References

- Nunes da Silva, I., Hernane D., Andrade, R., Bartocci, L., and dos Reis, S. *Artificial Neural Networks. A Practical Course*, Springer, 2017.